Using Hashed Storage in Dictionaries to Avoid or Reduce the use of nested loops

Problem Statement

We have two lists, list1 and list2

We would like to know the indices in each list of any value that is present in both

We can do this by looping through the list twice, one loop per index and noting all the cases where the values in the two lists match.

These entails looping through all the values in list2 for each value in list1

In [13]:
```
list1 = [3,5,10,3,7,4,6]
list2 = [3,4,2,10,3,8,9]
work_on2 = 0
for i in range(len(list1)):
    for j in range(len(list2)):
        work_on2+=1
        if list1[i] == list2[j]:
            print("Both elements are the same. " + '\nlist1 indice is: '+ st
```

```
Both elements are the same.
list1 indice is: 0
 list2 indice is: 0
Both elements are the same.
list1 indice is: 0
 list2 indice is: 4
Both elements are the same.
list1 indice is: 2
 list2 indice is: 3
Both elements are the same.
list1 indice is: 3
 list2 indice is: 0
Both elements are the same.
list1 indice is: 3
 list2 indice is: 4
Both elements are the same.
list1 indice is: 5
 list2 indice is: 1
```

Hashed Storage to avoid nested loops

We will use hashed storage (a python dictionary) to reduce the number of steps needed

First we create a dictionary that indicates whether each item in list1 is also in list2 The key of the dictionary is the index in loop 1, the value is true or false depending on

whether the value at that index is in list2.

Then we have a loop that goes through the key, value pairs. For the cases where the list1 value is in list2, we loop through loop2 to find the matching index

How does this save time? We have stored the test result in the dictionary, and we only have to look for the matching index in list2 if there is a match

In [14]:
```
from collections import defaultdict
work_on   = 0
hash_map = defaultdict(int)



for i in range(len(list1)):
    work_on += 1
    hash_map[i] = list1[i] in list2

for key, value in hash_map.items():
   if value == True:
        for j in range(len(list2)):
            work_on += 1
            if list1[key] == list2[j]:
                print("Both elements are the same. " + '\nlist1 indice is: '
   else:
        work_on += 1
```

```
Both elements are the same.
list1 indice is: 0
 list2 indice is: 0
Both elements are the same.
list1 indice is: 0
 list2 indice is: 4
Both elements are the same.
list1 indice is: 2
 list2 indice is: 3
Both elements are the same.
list1 indice is: 3
 list2 indice is: 0
Both elements are the same.
list1 indice is: 3
 list2 indice is: 4
Both elements are the same.
list1 indice is: 5
 list2 indice is: 1
```

In [15]:
```
print('nested loop number of steps: ' + str(work_on2))
print('hash map number of steps: ' + str(work_on))
```

```
nested loop number of steps: 49
hash map number of steps: 38
```

Question/Action

Can you estimate the order of the first approach, if the loop sizes are M and N?

The nested loop runs M times for the outer loop and N times for the inner loop, giving an order of O(MN).

For the second approach, the number of loops depends on slightly different factors. What would be the order if there are no matches in the lists? What would be the order if all values in the two list had a match?

If there are no matches, the inner loop never runs, so the order is O(M). If all values match, the inner loop runs for each element, giving O(MN) in the worst case.

Question/Action

Add the timing measurement tools from the other pair programming exercises to this notebook to get the elapsed time as well as the number of steps. The method using datetime is probably easier. Time both versions of the calculation.

Which is actually more important, the elapsed time or the order?

In [20]:
```
def nested_version(list1, list2):
    steps_nested = 0
    for i in range(len(list1)):
        for j in range(len(list2)):
            steps_nested+=1
            if list1[i] == list2[j]:
                pass
    return steps_nested

nested_version(list1, list2)
```

Out[20]: 49

In [28]:
```
# Elapsed time for nested
start_time = datetime.now()
step_nested = nested_version(list1, list2)
end_time = datetime.now()
elapsed_time = end_time - start_time
print(f"Cell ran in {elapsed_time}")
print('nested loop number of steps:', step_nested)
```

```
Cell ran in 0:00:00.000057
nested loop number of steps: 49
```

In [29]:
```
def hash_version(list1, list2):
    steps_hash   = 0
    hash_map = defaultdict(int)
```

```
    for i in range(len(list1)):
        steps_hash += 1
        hash_map[i] = list1[i] in list2

    for key, value in hash_map.items():
       if value == True:
            for j in range(len(list2)):
                steps_hash += 1
                if list1[key] == list2[j]:
                    pass
       else:
            steps_hash += 1
    return steps_hash

hash_version(list1, list2)
```

Out[29]: 38

In [31]:
```
# Elapsed time for hash
start_time = datetime.now()
step_hash = hash_version(list1, list2)
end_time = datetime.now()
elapsed_time = end_time - start_time
print(f"Cell ran in {elapsed_time}")
print('hash map number of steps:', step_hash)
```

```
Cell ran in 0:00:00.000049
hash map number of steps: 38
```

In [34]:
```
# Naive approach

words = ["apple", "banana", "orange", "apple", "grape"]

has_duplicate = False
for i in range(len(words)):
    for j in range(i + 1, len(words)):
        if words[i] == words[j]:
            has_duplicate = True
            break
    if has_duplicate:
        print(words[i])
        break

print("Has duplicate:", has_duplicate)
```

```
apple
Has duplicate: True
```

In [35]:
```
# this version uses hash storage in a set to avoid the double loop

words = ["apple", "banana", "orange", "apple", "grape"]

seen = set()
has_duplicate = False

for word in words:
    if word in seen:
```

```
        has_duplicate = True
        print(word)
        break
    seen.add(word)

print("Has duplicate:", has_duplicate)
```

```
apple
Has duplicate: True
```

Question/Action

Determine how long these two algorithms take to run, using the datetime method.

What happens if you increase the lenth of the list of words to 20?

In [55]:
```
def naive_approach(words):
    has_duplicate = False
    for i in range(len(words)):
        for j in range(i + 1, len(words)):
            if words[i] == words[j]:
                has_duplicate = True
                break
        if has_duplicate:
            #print(words[i])
            break

# Elapsed time for naive approach
start_time = datetime.now()
naive_approach(words)
end_time = datetime.now()
elapsed_time = end_time - start_time
print(f"Cell ran in {elapsed_time}")
```

```
Cell ran in 0:00:00.000042
```

In [56]:
```
def hash_approach(words):
    seen = set()
    has_duplicate = False

    for word in words:
        if word in seen:
            has_duplicate = True
            #print(word)
            break
        seen.add(word)

# Elapsed time for hash approach
start_time = datetime.now()
hash_approach(words)
end_time = datetime.now()
elapsed_time = end_time - start_time
print(f"Cell ran in {elapsed_time}")
```

```
Cell ran in 0:00:00.000052
```

In [59]:
```
# increase list by 20
words = [
    "apple","banana","orange","grape","pear",
    "peach","plum","mango","kiwi","lemon",
    "lime","cherry","melon","papaya","fig",
    "date","apricot","guava","apple","berry"
]
# Elapsed time for naive approach
start_time = datetime.now()
naive_approach(words)
end_time = datetime.now()
elapsed_time = end_time - start_time
print(f"Cell ran in {elapsed_time}")
```

```
Cell ran in 0:00:00.000046
```

In [60]:
```
# Elapsed time for hash approach
start_time = datetime.now()
hash_approach(words)
end_time = datetime.now()
elapsed_time = end_time - start_time
print(f"Cell ran in {elapsed_time}")
```

```
Cell ran in 0:00:00.000057
```

When the list size increases, both algorithms take longer to run. The naive approach should increase at a much faster rate because it uses nested loops and has quadratic time complexity, $O(n^2)$. The hash-based approach should increase more slowly because it runs in linear time, $O(n)$.

In [ ]:
```
```